

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO
a.a 2025/2026
Prof. Alessandro Ricci

[module 1.4]

ELEMENTS OF CONCURRENT PROGRAM DESIGN

GUIDELINES FOR DESIGNING CONCURRENT PROGRAMS

- Process organised in two macro steps
 - **step #1 - analysis**
 - identifying tasks
 - **step #2 - design**
 - concurrent architectures

STEP #1 - ANALYSIS

Non tutto può essere concorrente. Se per fare una torta devi prima rompere le uova e poi montarle, queste due azioni sono sequenziali. Se però mentre monti le uova un'altra persona prepara la teglia, quelle sono azioni concorrenti. L'analisi serve a capire quali parti del programma sono come "montare le uova" e "preparare la teglia".

- Analyzing the problem to identify exploitable concurrency

- ↳ identificare la suddivisione dei task e dei dati e le relative dipendenze
 - identifying **task & data decomposition** and their **dependencies**

Ci sono due definizioni di Task

- Task definition

- 1 – sequence of instructions that operate together as a group

- corresponding to some logical part of an algorithm or a program

Definizione "Implementativa" (Il Task come Gruppo di Istruzioni)
Qui guardiamo il task dal basso verso l'alto (bottom-up).

- 2 – a specific piece of work required to be done as a duty or responsibility of some agent

- more abstract / design-oriented definition

Definizione "Astratta" (Il Task come Responsabilità di un Agente)
Questa è la visione più evoluta, orientata al design (top-down). Qui il task non è solo "codice", ma un ruolo.

STEP #2 - DESIGN

Lo Step 2 trasforma l'analisi logica in una struttura operativa. Qui non stiamo più solo smontando il problema, ma stiamo costruendo la "macchina" che lo risolverà.

Nello step precedente abbiamo definito il task come una "responsabilità". Ora assegniamo quella responsabilità a un Agente. In termini tecnici, un agente può essere un Thread, un Processo o un Actor (nel modello ad attori) che incapsulano la logica. L'agente "sa" come eseguire il task e ne controlla l'esecuzione (quando iniziare, come gestire gli errori, quando finire). L'agente è l'entità che "vive" e consuma cicli di CPU.

• Choosing a suitable **concurrent architecture**

— assigning tasks to “agents”

- *active components* responsible of performing tasks
- encapsulate the logic & control of task execution

— in order to work together, agents share and use some *passive components*

- *shared resources* encapsulating result of agent work
- *coordination media/resources* to shape agent interaction and manage task dependencies

Gli agenti non vivono in isolamento; devono collaborare. Per farlo, usano componenti che non "fanno" nulla da soli, ma contengono dati o regole.

- Risorse Condivise sono i contenitori dei risultati.

> Esempio: Un buffer di memoria, una tabella di un database, o una variabile globale che accumula un conteggio.

- Risorse di Coordinamento sono gli strumenti che servono a gestire le dipendenze identificate nell'analisi. Senza di questi, avresti race conditions.

> Esempio: Semafori, Mutex, Monitor, Message Queues o Barriere.

ACTIVE VS. PASSIVE COMPONENTS DISCIPLINE

- Strong partition
 - **active components** (“agents”)
 - encapsulating a thread of control
 - no public interface Nessuno può "chiamare" un metodo di un agente. Non puoi dire a un thread: agente.faiQualcosa(). Se potessi farlo, saresti tu a controllare il suo tempo, rompendo l'incapsulamento del controllo.
 - every interaction between agents is enabled and mediated by shared passive components Se l'Agente A vuole comunicare con l'Agente B, non può bussare alla sua porta. Deve lasciare un "messaggio" in un componente passivo comune.
 - **passive components** il componente passivo non fa nulla da solo. "Dorme" finché un agente non lo invoca.
 - designed as monitors
- No hybrid entities Perché sono vietate? Sono la fonte principale di Race Conditions. Se un thread esterno chiama un metodo pubblico dell'oggetto mentre il thread interno dell'oggetto stesso sta modificando i propri dati, il sistema può corrompersi o andare in deadlock. La disciplina ti costringe a separare queste due nature.
 - an hybrid entity can be designed as a set of one or more active and passive components

STEP #1 - PROBLEM ANALYSIS

- **Objective:** identificare la suddivisione dei tasks e dei dati e le relative dipendenze
 - identifying task / data decomposition and their dependencies
- Two steps
 - 1 – ^A**Task** and ^B**Data** Decomposition Analysis
 - 2 – Dependency Analysis

1A

TASK DECOMPOSITION PATTERN

→ Prima identifichi le azioni (i Task), poi capisci quali dati servono a ogni azione.

- Following a *divide-at-impera* principle, the problem can be ^{suddiviso} naturally decomposed into a collection of independent or ^{quasi} ~~nearly~~ independent tasks

Se i task passano più tempo a comunicare tra loro che a lavorare, la concorrenza è inutile. L'overhead (il costo di gestione) deve essere minimo rispetto al tempo di calcolo.

- tasks must be sufficiently independent ^{affinchè} so as that ^{la gestione delle dipendenze} managing dependencies takes only a small fraction of the program's overall execution time
- First tasks are identified, ^{poi si stabilisce come} then how data can be decomposed ^{in base alla} given the task decomposition
 - in this case, data decomposition ^{viene dopo} follows task decomposition

- Example

└ producers/consumers systems

→ Esempio (Produttore/Consumatore):
Task 1: Leggere dati dal sensore.
Task 2: Elaborare i dati.
Task 3: Scrivere il log.

Qui il design segue le funzioni: la decomposizione dei dati è una conseguenza (i dati "fluiscono" tra i task).

18

DATA DECOMPOSITION PATTERN

Prima dividi i dati in pezzi (chunks), poi applichi lo stesso task (o una sequenza di task) su ogni pezzo.

- Problem's data can be easily decomposed into units that possono essere elaborate in modo relativamente indipendente ~~can be operated on relatively independently~~
 - first data units are identified, then task related to that units can be identified
 - in this case, task decomposition viene dopo ~~follows~~ data decomposition
- Scale well with the available number of processors
- Example

Scalabilità: È il punto di forza. Se hai 4 processori, dividi i dati in 4. Se ne hai 1000, li dividi in 1000. Il programma "scala" quasi linearmente con l'hardware.

matrix multiplication



Esempio (Moltiplicazione di Matrici):

Hai due matrici $C = A \times B$.

Dividi la matrice C in sotto-blocchi.

Ogni processore calcola un blocco indipendentemente dagli altri.

Il "task" è lo stesso per tutti, cambiano solo le coordinate dei dati.

2 DEPENDENCY ANALYSIS

La Dependency Analysis ti dice quanto il tuo problema è "parallelizzabile". Se scopri che ogni task dipende da quello precedente (una lunga catena), allora la programmazione concorrente non ti serve a nulla.

Perché si fa questa analisi? L'analisi serve a costruire il Grafo delle Dipendenze. Visualizzandolo, il progettista cerca di fare tre cose:

- Trovare il "Percorso Critico": È la sequenza di task dipendenti più lunga. Quella sequenza determina il tempo minimo di esecuzione. Se il percorso critico dura 10 secondi, anche se hai 1000 processori, il programma non durerà mai meno di 10 secondi.
- Massimizzare il Parallelismo: Identificare i task che non hanno dipendenze tra loro. Questi possono essere eseguiti contemporaneamente senza problemi.
- Ridurre l'Overhead: Se due task dipendono troppo l'uno dall'altro (si scambiano continuamente dati), forse è meglio unirli in un unico task più grande per evitare i costi della sincronizzazione.

Una volta decisi i task, devi capire come "incastrarli". Se sbagli questa fase, rischi il deadlock (tutti fermi) o le race conditions (dati corrotti).

Analyzing dependencies among the tasks within a problem's decomposition

- to **group** and **order** tasks ^{in base al} according to the type of dependencies
- ^{per} in order to maximize performances

• Possible kinds of dependencies

- **temporal** dependencies
- "Il Task B può iniziare solo dopo che il Task A è finito". È un vincolo di ordine.

- **data / resource** dependencies
- "Il Task B ha bisogno del risultato calcolato da A". Se A non ha ancora scritto il valore, B deve aspettare (Sincronizzazione).

• A classification of dependencies in coordination problem can be found in [MAL-93]

- interdisciplinary issue

1. Dopo aver definito i task, si mappano le relazioni esistenti, che si dividono in due tipi: Temporali (Il task B può iniziare solo dopo la fine del task A) e Dati / Risorse (Il task B richiede un'informazione o risorsa prodotta dal task A).
2. Compresa le dipendenze, l'obiettivo è organizzare i tasks in modo intelligente:
 - Raggruppamento: Accorpa i task strettamente sequenziali o troppo piccoli in un unico blocco. Evita che il costo di comunicazione tra processori superi il vantaggio del calcolo.
 - Ordinamento: Definisce la gerarchia di esecuzione. I task indipendenti vengono eseguiti in parallelo; quelli vincolati vengono messi in coda.
3. Risolvere le dipendenze a monte serve a prevenire i due problemi principali della concorrenza:
 - Race Conditions: Un ordine corretto impedisce a più task di modificare lo stesso dato contemporaneamente, evitando corruzioni.
 - Idle Time: Un buon bilanciamento del carico evita che un processore lavori al 100% e gli altri rimangano fermi ad aspettarlo.

STEP #2 - DESIGN LEVEL - SOME CONCEPTUAL CLASSES

Scelgo la miglior architettura concorrente in base al tipo del Problema definito durante la fase di Analisi

L'architettura concettuale scelta detta il punto di vista con cui viene strutturato e governato il parallelismo del sistema.

- From analysis to design: basic architectural classes proposed in literature [CAR-89]

1 – Result parallelism

Ogni agente è responsabile di creare una parte specifica del risultato finale del programma.

- ~~considerare il~~ envisioning parallelism in terms of program's *result*

2 – Specialist parallelism

Crei una rete di specialisti che comunicano tra di loro. Il parallelismo nasce dal fatto che tutti gli esperti sono "in ufficio" contemporaneamente.

- ~~considerare il~~ envisioning parallelism in terms of an ensemble of specialists (the *crew*) that collectively constitute the programs *gruppo*
insieme

3 – Agenda parallelism

Hai un'agenda di task (un "To-Do list"). Hai una squadra di agenti che sanno fare tutto. Quando un agente è libero, prende il primo task disponibile nell'agenda e lo esegue.

- ~~considerare il~~ envisioning parallelism in terms of program's *agenda of tasks*

Differenza tra Result, Specialist e Agenda Parallelism? La differenza riguarda la logica con cui si suddivide e coordina il lavoro fra un insieme di agenti (processi, worker), cercando di parallelizzare il più possibile le attività.

- Nel caso "Result" la suddivisione viene identificata a partire dal risultato che si vuole ottenere. Ad esempio, consideriamo il problema della moltiplicazione di due matrici quadrate. Si osserva che ogni elemento della matrice risultato può essere computato indipendentemente, per cui si assegna quel task ad un agente diverso.

- Nel caso "Specialist", la suddivisione avviene identificando un pool di agenti ognuno in grado di svolgere un compito specifico. Ad esempio, in un processo di elaborazione di immagini a più stadi, ogni agente è specializzato nell'applicare l'elaborazione di uno stadio.

- Nel caso "Agenda", la suddivisione avviene invece fra un pool di agenti in grado di compiere qualsiasi task (come "generalisti") e l'elaborazione concorrente avviene a partire da un'agenda/workflow che identifica i vari task da fare e le loro dipendenze. Gli agenti eseguono man mano i task da fare.

1

RESULT PARALLELISM

Qui il centro di tutto è l'oggetto finale. Immagina di voler produrre un risultato complesso (come una matrice) e di dividerlo in pezzi. Il Result Parallelism è un approccio in cui il punto di partenza del design non è "cosa devo fare", ma "com'è fatto l'oggetto finale".

"ottenere il parallelismo" (dall'inglese "we get parallelism") significa raggiungere l'esecuzione simultanea di più operazioni sfruttando l'hardware a disposizione (i vari core della CPU).

- Designing the system *around the data structure or resource* ^{ottenuta} ~~yielded~~ ^{final} ~~as the ultimate~~ result
 - ^{si ottiene il} ~~we get~~ ^{calcolando} parallelism ^{contemporaneamente} ~~by computing~~ all the elements of the result ~~simultaneously~~
 - strictly related to the data-decomposition pattern
- Each agent is assigned to produce one piece of the result
 - they all work in parallel ^{entro i limiti naturali} ~~up to the natural restriction~~ imposed by the problem

L'agente lavora in parallelo agli altri. Non gli interessa cosa fanno gli altri, finché ha i dati necessari per calcolare il suo pezzetto. Il parallelismo è limitato solo dalla logica. Ad esempio, se per calcolare il pezzo B serve che il pezzo A sia finito, l'agente B dovrà aspettare (dipendenza).
- Proper shared data structures are designed to wrap the result (data) under construction
 - encapsulating mutex and synchronization issues

RESULT PARALLELISM:

“BUILDING A HOUSE” EXAMPLE

- Think about the components of the house
 - front, rear, side walls, interior walls, foundations, roof, etc.
- Proceed by building all the components simultaneously
 - assembling them as they are completed (i vari componenti prodotti vengono mano mano assemblati quando sono completati)
- Use separate agents (workers) set to work on the different parts
 - proceeding in parallel up to the point where work on one component can't proceed until another is finished

2 SPECIALIST PARALLELISM

Lo Specialist Parallelism è un approccio al design in cui il sistema viene modellato come una squadra di esperti, ognuno con un compito unico e specifico. Se nel Result Parallelism dividevamo il risultato finale, qui dividiamo la funzione macro.

- Designing the system ^{insieme} around an ~~ensemble~~ of specialists connected into a logical network of some kind
 - parallelism ^{deriva dal fatto che} results from all the nodes of the logical network (all the specialists) being active ^{contemporaneamente} simultaneously
- Each agent is assigned to perform one specified kind of task
 - they all work in parallel ^{entro i limiti naturali} up to the natural restriction imposed by the problem
- ^{Gli strumenti di coordinamento} ~~Coordination means~~ (communication protocols, shared data structures) are used to support specialists communication and coordination
 - examples: message boxes, blackboards, event services
- Opposite approach with respect to result parallelism

Significa che se lo Specialista B è lentissimo, Specialista A finirà per stare fermo ad aspettare. In questa architettura, il sistema è veloce quanto lo specialista più lento (bottleneck).

Poiché ogni specialista fa una cosa diversa, devono parlarsi costantemente.

Ideale per sistemi con funzioni molto diverse (Task Decomposition).

SPECIALIST PARALLELISM: “BUILDING A HOUSE” EXAMPLE

- Identify the separate skills required to build the house
 - surveyors, excavators, foundation builders, carpenters, roofers, etc
 - different kind of tasks
- Then assemble a constructor crew in which each skill is represented by a separated specialist worker
- They all start simultaneously
 - but initially most workers will have to wait around
 - once the project is well underway, however, many skills (hence many workers) will be called to play simultaneously
- Remarks
 - strong role of worker interaction and coordination
 - different kind of strategies
 - e.g. pipelined jobs

- Coordinamento: Fondamentale. Poiché l'idraulico non può mettere i tubi se il muratore non ha fatto il muro, devono comunicare tra di loro.

- All'inizio molti aspetteranno (l'elettricista aspetta il muratore), ma una volta avviato il cantiere, avrai persone diverse che lavorano contemporaneamente in stanze diverse facendo cose diverse.

AGENDA PARALLELISM

Se il Result Parallelism si focalizza sul "pezzo di puzzle" da consegnare e lo Specialist Parallelism si focalizza sul "mestiere" dell'agente, l'Agenda Parallelism si focalizza sulla lista delle cose da fare (Non importa chi fa cosa, l'importante è svuotare la lista il prima possibile.).

L'agenda non è altro che una sequenza di attività (o un "To-Do list"). Il sistema è progettato definendo i vari passi necessari per arrivare alla soluzione.

- Designing the system around a particular **agenda of tasks** (activities in [CAR-89]) and then assign many workers to each step
 - and also possibly carry on different steps in parallel
 - *workflow*
- Each agent is assigned to ~~help out with the~~ current item on the agenda

si occupa del

 - they all work in parallel ~~up to the natural restriction~~ imposed by the problem

Qui gli agenti sono interscambiabili. Non abbiamo l'idraulico o l'elettricista; abbiamo un team di operai "tuttofare".

entro i limiti naturali
- Shared data structures are designed to manage data consumed and produced by agenda activities
 - examples: bounded-buffers

AGENDA PARALLELISM:

“BUILDING A HOUSE” EXAMPLE

Si stila una lista delle cose da fare (un'agenda) strutturata in fasi logiche consecutive. Non si può cambiare l'ordine: la fase 2 inizia solo quando la fase 1 è completata.

All'interno di ogni singola fase (es. fare le fondamenta, tirare su i muri, fare il tetto), si mettono tanti operai a lavorare contemporaneamente sullo stesso obiettivo. Il parallelismo non nasce dal fare cose diverse, ma dal fare la stessa cosa in tanti per finire prima.

- We write out a sequential agenda and carry it in order, but at each stage we assign many workers to the current activity

– doing the foundation, doing the frame, doing the roof, etc

- We assemble a work team of **generalists**
 - each capable of performing any construction step
- The team is set to work stage by stage following the agenda

Tutta la squadra si sposta compatta da una fase all'altra dell'agenda. Quando si fanno le fondamenta, tutti scavano; quando si passa al tetto, tutti mettono le tegole.

REMARKS

Riassunto della Metodologia

- Analisi: Scomponi il problema (Task o Data decomposition).
- Design: Scegli una struttura (Result, Specialist o Agenda).
- Rifinitura: Se un componente è ancora troppo complesso, ricomincia dal punto 1 applicandolo solo a quel componente.

- The boundaries between the three classes are not rigid
 - we will often mix elements of several approaches in getting a particular job done
 - e.g. a specialist approach might make secondary use of agenda parallelism, for example, by assigning a team of workers to some speciality
- However the three approaches represent *three clearly separate ways of thinking about the problem*
 - result parallelism focuses on *the shape of the finished product*
 - specialist parallelism focuses on *the makeup of the work crew*
 - agenda parallelism focuses on *the list of tasks to be performed*
- The approach can be recursively applied
 - following task and data decomposition

forma

composizione

gruppo

da essere svolte

Puoi applicare lo Step #1 (Analisi) e lo Step #2 (Design) "dentro" un singolo task.

Esempio Ricorsivo:

- Livello 1 (Sistema): Usi lo Specialist Parallelism per dividere il programma in "Modulo Rete" e "Modulo Calcolo".
- Livello 2 (Dentro il Modulo Calcolo): Prendi quel modulo e lo analizzi di nuovo. Decidi che, per calcolare i dati, la strategia migliore è il Result Parallelism (dividere i dati in sottogruppi).
- Livello 3 (Dentro il singolo calcolo): Se il calcolo è ancora troppo complesso, puoi dividerlo ulteriormente usando un'agenda.

- Result Parallelism: Guardo l'oggetto: "Come divido questo output in pezzi?" (Focus sull'Oggetto).
- Specialist Parallelism: Guardo le persone: "Di quali esperti ho bisogno per questa catena di montaggio?" (Focus sulle Competenze).
- Agenda Parallelism: Guardo la lista: "Quali sono i passi da seguire uno dopo l'altro?" (Focus sul Processo).

USING CONCEPTUAL CLASSES - BASIC STEPS

- To write a concurrent program (adapted from [CAR-89]):
 - choose the concept class that is most natural for the problem;
 - write a program using the method that is most natural for that conceptual class; and
 - if the resulting program is not acceptably efficient, transform it methodically into a more efficient version by switching from a more-natural method to a more-efficient one

Non cercare subito la soluzione più veloce, cerca quella che si adatta meglio alla forma del problema

A volte, il modello più naturale non è il più performante su una specifica macchina. Invece di buttare tutto, trasformi il design seguendo una logica.

Da Specialist ad Agenda: Ti accorgi che uno "Specialista" è sempre sommerso di lavoro mentre gli altri dormono? Trasformalo in un'Agenda dove più lavoratori generici possono dare una mano a smaltire quel compito specifico.

Una volta scelta la classe, implementala seguendo le sue regole standard:

Se sei in Specialist, crea i tuoi agenti e falli comunicare tramite canali o messaggi.

Se sei in Agenda, crea un Worker Pool e una coda di task.

Non cercare scorciatoie in questa fase. L'obiettivo è la correttezza.

Se il problema riguarda la produzione di un oggetto complesso fatto di parti: usa Result Parallelism.

Se il problema riguarda un flusso di lavoro con competenze diverse: usa Specialist Parallelism.

Se il problema riguarda una massa di piccoli compiti simili: usa Agenda Parallelism.

STEP #2: CONCURRENT ARCHITECTURES

- Some specific architectures can help to ^{colmare il divario} bridge the gap between the conceptual class and the implementation (adapted from [MAG-99][PAT-06])
 - 1 – **Master-Workers (or Manager-Workers)** ^{È la traduzione pratica dell'Agenda Parallelism.}
 - A • Task-oriented frameworks ^{Si basa su framework che mettono al centro il concetto logico di "Task" separato da quello fisico di "Thread"}
 - B • Fork-Join variant ^{Una variante ricorsiva in cui i Worker possono diventare a loro volta Master, creando nuovi sotto-task per implementare strategie parallele di tipo divide-and-conquer}
 - 2 – **Filter-Pipeline** ^{È la traduzione pratica dello Specialist Parallelism.}
 - Stream-based patterns
 - 3 – **Shared-Space / Blackboard** ^{Introduce un modello di coordinamento basato sullo spazio anziché sui canali diretti: Gli agenti non comunicano direttamente tra loro, ma interagiscono tramite una "bacheca" o spazio comune condiviso.}
 - Space-based coordination patterns
 - 4 – **Announcer-Listeners** ^{È il classico modello a notifiche push, noto anche come Publisher-Subscriber o Observer concorrente.}
 - Event-based coordination patterns

1 MASTER-WORKERS

- Description

- the architecture is composed by a *master* agent and a possibly dynamic set of *worker* agents, interacting through a proper coordination medium functioning as a **bag of tasks**

Componente Attivo

- **master agent** (non è detto che sappia la quantità di worker a disposizione: quantità dinamica di worker)

- ^{suddivide} **decompose** the global task in subtasks
- **assign** the subtasks by inserting their description in the bag
- **collect tasks result** Aspetta che i lavoratori finiscano e mette insieme i risultati per formare la soluzione finale.

Componente Attivo

- **worker agent** I Worker sono agenti identici

- **get the task to do from the bag, execute the tasks and communicate the results** Una volta finito un compito, torna subito alla borsa a cercarne un altro.

- variant: proper monitor data structures can be used to aggregate results

- **bag of tasks resource** È il componente passivo che media tutto.

- typically implemented as a blackboard or a bounded buffer

- Example of problems

- quadrature problem

TASKS AND ASYNC PROGRAMMING

Visione del problema/soluzione in termini di Task, non Thread

- The execution of a task ^{avviene in modo asincrono} ~~occurs asynchronously~~ ^{rispetto al} ~~with respect to~~ the control flow that created and submitted the task ^{flusso di controllo di chi ha}

- already discussed in previous module

- Example: Java Executor framework**

- master-worker architecture

- a task is submitted to an executor ^{tramite} ~~by means of~~ an execute or submit method, which returns immediately

ha uno stato (il risultato) che cambia nel tempo

- a *future* can be returned

L'oggetto Future (implementato come Monitor) è il ponte che permette al Master di recuperare i pezzi del risultato prodotti dai Workers in modo asincrono. Rappresentano il risultato futuro (quando fai submit, mi restituisci un Future: posso sapere se risultato è arrivato (op. asincrona) o bloccarmi in attesa del risultato (op. sincrona))

- the task is *asynchronously* executed by one of the executor threads, according to the executor policy

Quando invochi `executor.submit(task)`, non stai dando il via all'esecuzione vera e propria, stai facendo un'operazione di consegna.

- Submit: Tu (il Thread Principale) consegna il compito all'Executor.

- Messa in Coda: L'Executor prende il task e lo infila in una bag of tasks.

- Sganciamento: Una volta che il task è al sicuro nella bag, l'Executor ti dice: "Ok, ricevuto! Vai pure avanti con le tue cose, me ne occupo io". Ed è qui che il metodo ritorna immediatamente.

- Esecuzione Asincrona: Solo in questo momento, uno dei Worker (i thread del pool) che era "addormentato" si sveglia, vede il nuovo compito nella coda, lo prende e inizia a lavorarci.

Il task ha senso a livello logico non più fisico (rappresenta un'unità di lavoro da fare e ben definito che si presuppone che inizi e termini, non un Thread. Task viene definito in base alla strategia con cui vogliamo risolvere il problema). Executor è un componente passivo che all'interno gestisce dei Thread, non è un Thread.

- L'ExecutorService (Il Master): Rappresenta il gestore del sistema. È lui che riceve i compiti (`submit()`), decide come organizzarli e li distribuisce.

- La BlockingQueue (L'Agenda): È la lista dei task da eseguire. Quando fai `executor.submit(task)`, il task viene messo in questa coda "bacheca".

- I Thread nel Thread Pool (I Worker): Sono gli agenti attivi. Questi thread sono creati dall'executor e restano in attesa (in loop) finché non c'è qualcosa nell'Agenda (la coda). Quando un thread finisce un compito, torna automaticamente alla coda per prenderne un altro.

- Runnable / Callable (I Task): Rappresentano l'unità di lavoro specifica definita dall'utente.

1A

TASK-ORIENTED FRAMEWORKS

- **Task concept**

- abstract, discrete, independent *unit of work*

- domain/problem level

- uncoupled from the notion of thread

Il task esiste anche se non c'è un thread pronto a eseguirlo. Rimane lì, nella "Bag of Tasks", finché qualcuno non lo prende.

- it will be executed by some thread or process of the system

- **Task-oriented analysis & design**

- analyzing the problem to identify exploitable concurrency

- identifying task & data decomposition and their dependencies

- **Tasks as programming abstraction**

- higher-level than threads

- logic concurrency vs. physical concurrency

- frameworks/libraries example: Java Executors

- Abstract: Non ti preoccupi se verrà eseguito da una CPU Intel, un core di uno smartphone o un server remoto.
 - Discrete: Non è un flusso infinito di dati, ma un'unità di lavoro con i suoi input specifici e (solitamente) un output. È una "porzione" atomica di un algoritmo.
 - Independent: Ogni task dovrebbe essere un'entità a sé stante, con i suoi dati di input e il suo obiettivo.

- Logica: È la struttura del tuo programma. Se hai diviso il lavoro in 100 task che potrebbero girare insieme, il tuo programma è logicamente concorrente.
 - Fisica: È quello che succede davvero nell'hardware. Se il PC ha solo 2 core, quei 100 task verranno eseguiti 2 alla volta.

Il vantaggio? Se scrivi un programma orientato ai task, esso scalerà automaticamente: girerà su un vecchio PC a 2 core (piano) e su un supercomputer a 128 core (velocissimo) senza che tu debba cambiare una singola riga di codice.

TASKS AND ASYNC PROGRAMMING

- In task-oriented frameworks, there are two distinct roles:
 - who *submits* the task
 - playing the role of master
 - who *executes* the task
 - worker threads inside the framework
- Key point
 - the task is executed **asynchronously** with respect to the control flow of the task submitter
 - *task submission does not block waiting the task to be completed*
l'invio del compito è asincrono. In termini tecnici, il thread che invia il lavoro (chiamiamolo "Thread Utente") non deve restare "congelato" sulla riga di codice dell'invio finché il lavoro non è finito.
 - follow-up problem: *how the master can collect the task results (computed asynchronously)?*
 - Option 1 • through a bag of results
 - Option 2 • through a specifically-designed collector/aggregator monitor
 - Option 3 • **future mechanism**

FUTURE MECHANISM

- Future mechanism
 - tramite l'esecuzione di by issuing an asynchronous computation (task), an object *representing the future result* or the state of the computation itself is created and immediately returned
- The object could provide operation for
 - checking the state of the task (*poll*)
 - blocking when the task result is needed
 - cancel the ongoing task, when possible
 - catching errors/exception related to the ongoing task
- **Important remark**
 - with task+future, we have async programming *but still allowing for blocking behaviour*
 - in event-driven programming (next slides) blocking behaviour is no more allowed

TASKS & VIRTUAL THREADS

- Tasks promote a higher-level logical view about concurrency
 - vs. physical view given by (OS, physical) threads
- Actually, a similar logical view is provided by frameworks/languages/platforms introducing a logical/lightweight notion of thread
 - also called user-level threads / fibers / green threads
 - uncoupled from physical threads (OS-level), used to run them
- In Java, this logical view is represented by **virtual threads** recently introduced (as a preview) in JDK 19
 - Java Enhancement Proposal (JEP) 425
 - <https://openjdk.org/jeps/425>
 - a main result of the project Loom
 - <https://openjdk.org/projects/loom/>
- Discussed in a module-lab

TASKS & STRUCTURED CONCURRENCY

- Simplify concurrent programming by introducing an API for **structured concurrency**

considera
– treats groups of related tasks, running in different threads, as a single unit of work

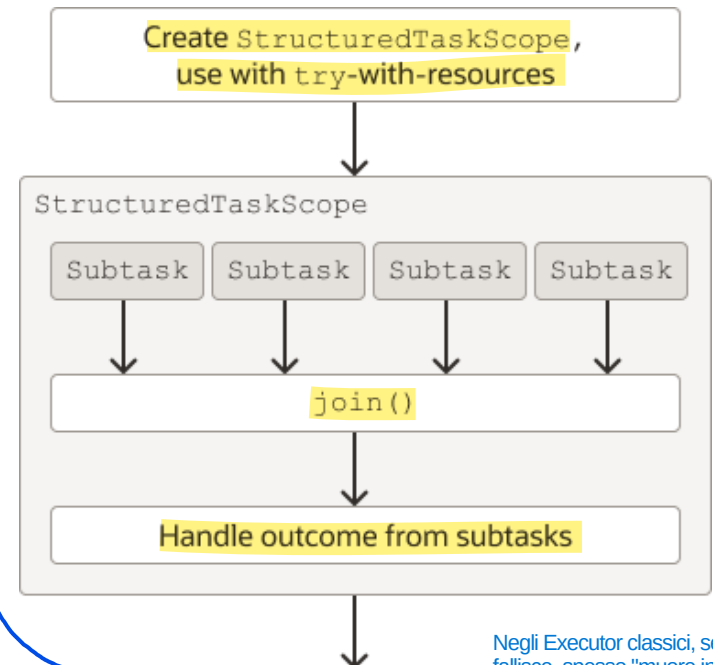
ottimizzazione del
– streamlining error handling and cancellation

– improving reliability, and enhancing observability

Se con l'Executor Service avevamo un "Master" che lanciava task e se ne dimenticava ("fire and forget"), con la Structured Concurrency il Master si prende la responsabilità dell'intero gruppo di task. Nella programmazione concorrente classica, se un metodo lancia tre thread per fare tre calcoli, quei thread "scappano" via. Se il metodo principale termina o crasha, i tre thread potrebbero continuare a girare come "thread zombie", sprecando risorse. Con la Structured Concurrency, i task correlati vengono legati a un ambito (scope). Il codice non può uscire da quell'ambito finché tutti i task figli non sono terminati.

- Proposed by Java Enhancement Proposal (JEP) 425
 - <https://openjdk.org/jeps/425>

Uno dei problemi più grandi del parallelismo è: "Se non mi serve più il risultato, come fermo tutti i thread che stanno ancora lavorando?" Con la Structured Concurrency, la cancellazione è propagata



Negli Executor classici, se un task fallisce, spesso "muore in silenzio" o devi gestire manualmente complicate catene di eccezioni.

Nella Structured Concurrency, se un sotto-task fallisce, lo Scope può decidere immediatamente cosa fare

13

FORK-JOIN

- Description
 - applying the master worker architecture recursively
 - workers are themselves masters, producing tasks and collecting results
 - effective approach to implement a concurrent divide-and-conquer strategies
 - can be realized either using ~~using~~ a bag of tasks or directly
- Example of problems
 - adaptive quadrature problem
 - merge-sort

si usa un'unica grande "Bag of Tasks" logica a livello di sistema, condivisa da tutti i worker, e non una nuova bag per ogni sotto-master ricorsivo.

Quando un thread decide di dividere il lavoro, istanzia direttamente un nuovo thread figlio e gli affida il sotto-compito. Il thread padre poi si mette in attesa diretta del risultato del figlio (fase di Join).

2 FILTER-PIPELINE

- **Description** il lavoro non viene diviso in "pezzi di dati" indipendenti, ma in fasi di elaborazione.
 - the architecture is composed by a linear chain of agents interacting through some *pipe* or *bounded buffer* or *channel* resources
 - **generator agent**
 - the agent starting the chain, generating the data to be processed by the pipeline
 - **filter agent** Ogni filtro riceve un dato, esegue una sola trasformazione specifica e lo passa immediatamente al filtro successivo.
Nota bene: Un filtro non sa cosa è successo prima e non sa cosa succederà dopo. Conosce solo il suo "input pipe" e il suo "output pipe".
 - an intermediate agent of the chain, consuming input information from a pipe and producing information into another pipe
 - **sink agent**
 - the agent terminating the chain, collecting the results
- Example of problems
 - image processing

3 SHARED-SPACE

rappresenta un approccio al parallelismo radicalmente diverso dalla filter-pipeline. Se la filter-pipeline è una catena di montaggio "rigida", lo Shared-Space è un ufficio postale o una bacheca condivisa.

- Description

- the architecture is composed by a set of agents communicating and ~~indirectly~~ interacting through a shared space, as a shared resource functioning as a blackboard-like coordination ~~media~~ ^{risorsa}
- the space-based coordination ~~media~~ ^{risorsa} provides operations to add new information items and to query/retrieve/remove them, eventually including some synchronisation feature

- Features

- data-driven coordination
- temporal uncoupling

In questo modello, non è il "comando" a far muovere il sistema, ma la presenza del dato. Un agente "Specialista" rimane in attesa (in stato di blocking) finché sulla bacheca non appare una tupla che corrisponde al suo profilo di ricerca.

Nello Shared-Space:

- L'agente A può produrre un dato alle 10:00 e terminare la sua esecuzione.
- L'agente B può svegliarsi alle 11:00, trovare quel dato sulla bacheca e lavorarlo.
- Conseguenza: Gli agenti non hanno bisogno di sapere l'uno dell'esistenza dell'altro, né di coincidere nel tempo.

- Main reference in literature

- Tuple spaces and Linda coordination language [GEL-85][GEL-92]

4

ANNOUNCER-LISTENERS

Se lo Shared-Space era una bacheca dove "chi voleva, prendeva", questo modello è un sistema di notifiche push.

- **Description** La caratteristica fondamentale di questo modello è che gli agenti non si conoscono minimamente tra loro. La comunicazione avviene attraverso un intermediario.
 - the architecture is composed by an *announcer* agent and a dynamic set of *listener* agents, interacting through an *event-service*
 - **announcer agent** È la sorgente degli eventi (componente attivo). Non sa chi siano i listener e non gli importa. Quando accade qualcosa (un sensore scatta, un utente clicca, un prezzo cambia), invia una notifica al servizio eventi.
 - announce the occurrence of events on the event service
 - **listener agents** È un insieme dinamico di agenti (possono aumentare o diminuire nel tempo). "Sottoscrivono" il loro interesse per certi tipi di eventi. Quando l'evento accade, eseguono il loro Event Handler (codice di risposta).
 - si registrano al register on the event service per ricevere notifiche quando so as to be notified of the si verificano occurrence of events di interesse interesting for the listener
 - **event-service resource** Architetturalmente è una risorsa passiva di coordinamento. Ha il compito di raccogliere gli eventi e smistarli a chi di dovere. La sua esistenza serve a disaccoppiare (uncouple) gli agenti attivi: chi genera l'evento non sa (e non gli interessa) chi lo riceverà, e viceversa.
 - uncouple announcer-listeners interaction
 - collect and dispatch events
- Example of problems
 - parallelization of event-driven applications
 - achieved by parallelizing the execution of event handlers

REVISITING THE MODEL VIEW CONTROLLER /1

Pattern Observer
- Model: sorgente di eventi
- View: observer degli eventi aggiornati dal model

Issue

- how to separate model / view / controller aspects in the design of complex concurrent programs

modello di dominio (classi con cui modello il dominio)

come rappresento all'utente il modello (per stesso model, posso avere più view). Ogni volta che il modello cambia, la view si dovrebbe aggiornare

A first mapping

- encapsulating control in agents

logic

- active components of the program
- encapsulating the control logic

gestisco l'input da parte dell'utente (l'input viene gestito attraverso costrutti che passano per la view). Il controller prende l'input dell'utente dalla view ed agisce sul model che se viene modificata viene aggiornata la view associata

- model as shared passive components

- can be used by agents
- encapsulating mutex properties
- includes coordination media

In un sistema concorrente, il Controller non è più solo una classe di passaggio, ma diventa un Agente. Ogni Controller ha un proprio "ciclo di vita" (spesso un thread dedicato). È l'unico che "prende decisioni" e incapsula la logica di controllo. Riceve stimoli (dagli utenti o da altri agenti), elabora la logica e decide come modificare il Modello o aggiornare la Vista. Essendo un agente autonomo, può gestire task in background senza bloccare gli altri componenti.

- view as shared passive components

- enabling and mediating the interaction with users
- observed and used by agents
- can access to model resources to get the state

Il Modello non è un agente (non "fa" cose da solo), ma è una risorsa che gli agenti usano. Poiché più Agenti (Controller) potrebbero voler modificare il Modello contemporaneamente, il Modello deve incapsulare i meccanismi di sincronizzazione (come i synchronized o i Lock in Java). Il Modello spesso agisce come lo Shared-Space (o Blackboard) che abbiamo visto prima. Diventa il luogo dove gli agenti lasciano dati affinché altri li leggano. È il custode dei dati "puri" del sistema.

Anche la View è passiva: non decide cosa mostrare, ma "abilita" l'interazione. Riceve gli input dall'utente (mouse, tastiera) e li rende disponibili agli Agenti Controller. Gli Agenti osservano la View per capire cosa vuole l'utente, e la View può accedere al Modello (spesso in sola lettura) per recuperare i dati da mostrare.

Perché questa separazione è importante?
In un programma sequenziale, il Controller chiama il Model, aspetta, e poi aggiorna la View.
In un programma concorrente:

- L'utente clicca un tasto (View).
- L'Agente Controller A riceve il comando e inizia un calcolo pesante.
- Nel frattempo, l'Agente Controller B aggiorna un'altra parte del Model (es. dati di rete).
- Il Model gestisce l'accesso di A e B tramite Mutex per evitare corruzione dei dati.
- La View riflette i cambiamenti man mano che avvengono, agendo come una finestra sullo stato condiviso.

REVISITING THE MODEL VIEW CONTROLLER /2

Problema: cambia qualcosa se siamo in ambito concorrente? Sì: la parte di model potrebbe contenere componenti attivi che lavorano sulle entità del dominio; la view è gestito tramite EDT (single thread)

Per design, tutta la parte reattiva che ha a che fare con la reazione ad eventi dell'interfaccia grafica la deve gestire EDT (quindi devo "tagliare" chi esegue il codice riguardante GUI e Business Logic, cioè EDT esegue codice GUI ed 1+ Thread il codice del Controller e Model: ciò aumenta la reattività perché ho più componenti attivi).

• Issues / problems (to be discussed)

- Separazione netta: Gli Agenti (Controller) calcolano la logica su thread separati.
- Delegazione (Marshalling): Quando un Agente vuole aggiornare la View, non può farlo direttamente. Deve "impacchettare" l'aggiornamento e inviarlo all'unico Agente GUI autorizzato.
- In Java: Si usa `SwingUtilities.invokeLater()`

A common design choice in existing GUI toolkit is to adopt a single active component to be responsible of managing the GUI aspects, with all GUI components are not thread-safe

un solo thread (Event Dispatch Thread - EDT) può toccare la grafica.

- how to deal with it using the approach described so far?
- More generally: MVC adopts an *event-based* approach to interactions

- *observer pattern*

- the original one is synchronous and involves control coupling

Il problema: Se il Modello cambia e notifica 10 Observer, lo fa chiamando i loro metodi uno dopo l'altro. Se un Observer è lento, blocca il Modello e tutti gli altri Observer. C'è un control coupling (accoppiamento di controllo) troppo forte.

- Rethinking the observer pattern for concurrent systems

- producer/consumer architecture
 - bounded-buffer is an event queue
 - producers: who generate events
 - consumer: who reacts and consumes events

BIBLIOGRAPHY

- **[CAR-89]** Nicholas Carriero and David Gelernter, "How to write parallel programs: a guide to the perplexed", ACM Computing Surveys (CSUR). Volume 21, Issue 3 (1989)
- **[GEL-92]** David Gelernter and Nicholas Carriero. 1992. Coordination languages and their significance. Commun. ACM 35, 2 (Feb. 1992), 97–107. DOI:<https://doi.org/10.1145/129630.129635>
- **[GEL-85]** David Gelernter. 1985. Generative communication in Linda. ACM Trans. Program. Lang. Syst. 7, 1 (Jan. 1985), 80–112. DOI:<https://doi.org/10.1145/2363.2433>
- **[MAT-05]** T. Mattson, B. Sanders, B. Massingill, "Patterns for Parallel Programming", Addison Wesley (2005)
- **[MAG-99]** Jeff Magee and Jeff Kramer, "Concurrency - State Models and Java Programs", Wiley (<http://www.doc.ic.ac.uk/~jnm/book/>)
- **[MAL-93]** Thomas W. Malone, Kevin Crowston. "The interdisciplinary study of coordination". Center for Coordination Science, Alfred P. Sloan School of Management, Massachusetts Institute of Technology, 1993
- **[PAT-06]** Jorge Luis Ortega-Arjona. Patterns for Parallel Software Design. Wiley (2006)